UD8. CREACIÓN DE CLASES Y PAQUETES.

1. Definición de una clase.
2. Miembros de una clase.
3. Diferencia entre módulo y método.
4. Métodos sobrecargados.
5. Crear un objeto o instancia.
6. Constructores.
7. Destrucción de objetos.
8. Asignación de objetos.

PROG. Nuria Fuentes Pérez

1. Definición de una clase

```
class nombreClase {
   // cuerpo de la clase
}
```

La sintaxis completa sería:

```
[acceso] [tipoDeClase] class nombreClase [superclase] [interface] {
   [lista de atributos]
   [lista de métodos]
}
```

1. Definición de una clase

- [acceso]
  - ✓ ***public***: puede ser usada por cualquier clase en cualquier paquete.
  - ✓Nada (paquete): sólo puede ser usada por las clases del mismo paquete.
- [tipoDeClase]
  - ✓***final***: es aquella que no puede tener clases que la herede.
  - ✓***abstract***: es una clase que puede tener herederos pero no puede ser instanciada. Sirven para modelar conceptos.

1. Definición de una clase

- [superclase]
  - ✓ ***extends*** *ClaseMadre*
- [Interface]
  - ✓ ***implements*** *Interface1[,Interface2]…*

2. Miembros de una clase

LISTA DE ATRIBUTOS:

[acceso][***static***][***final***] tipo *nVariable*[=valor];

LISTA DE MÉTODOS:

```
[acceso][static][abstract][final] tipoDevuelto nMétodo ([tipo1 nom1 [,tipo2 nom2]…])
                                        [throws excp1[,excp2]..] {
        [declaración de variables]
        [algoritmo]
        [return[(expresión)]
}
```

2. Miembros de una clase

- [acceso]
  - ✓ ***public***
  - ✓ ***private***
  - ✓ ***protected***
  - ✓ Nada

ACCESO DESDE:	private	protected	public	nada
La propia clase				
Subclase en el mismo paquete				
Otra clase en el mismo paquete				
Subclase en otro paquete				
Otra clase en otro paquete				

S	Puede acceder		
N	NO puede acceder		

2. Miembros de una clase

Para referirnos a un atributo o método de un objeto sería

NClase.*nMiembro* | *nObj.nMiembro*

ATRIBUTOS

- [***static***]: es un atributo de clase.
- [***final***]: es una constante

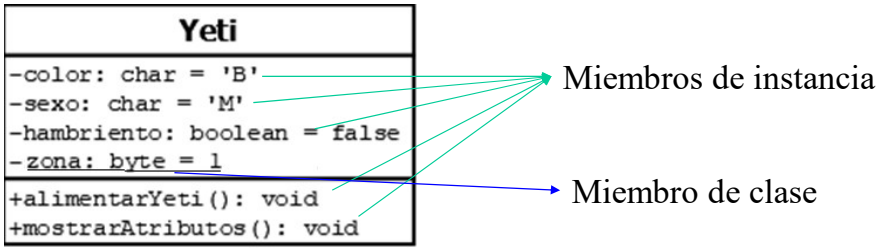
MÉTODOS

- [***static***]: métodos de clase, no de instancia. Ej. Leer.dato()
- [***abstract***]: método abstracto. Definido en subclase.
- [***final***]: método que no puede ser redefinido por ningún descendiente de la clase
- tipoDevuelto
- Parámetros se pasan todos por valor. (primitivos, objetos y arrays)

2. Miembros de una clase

- [*native*]
- [*sychronized*]
- [*throws*]

2. Miembros de una clase



Uso de miembro de clase

Yeti.zona

Uso de miembro de instancia

j.alimenarYeti(); //j es un objeto Yeti

3. Diferencia entre módulo y método

Los módulos que hemos visto son métodos de clase, por eso eran TODOS **static** y las llamadas se escribían **Nclase.nMétodo**
Los métodos de instancia, se diferencian de los de clase en que, además de recibir los parámetros, se recibe como mensaje un objeto de la clase.

public void alimenarYeti()➔es un método de instancia, es decir, aunque no recibe parámetros, recibe, como mensaje, un objeto Yeti.

Para referenciar al objeto de la clase enviado a un método de instancia, como mensaje, se usa la referencia **this**.

```
public void alimentarYeti() {
    if (this.hambriento) {
        System.out.println("Hum... un placer!");
        this.hambriento = false;
    } else {
        System.out.println("No, gracias...estoy lleno");
    }//Fin Si
}
```

4. Métodos sobrecargados.

Un método está sobrecargado cuando se define varias veces con distinto número de parámetros o con parámetros de distinto tipo en la misma clase.

Java no admite dos métodos iguales en la misma clase con distinto valor de retorno.

5. Crear un objeto o instancia

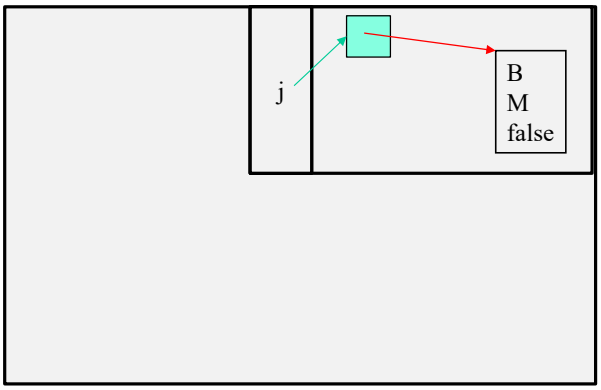
//Entorno:

tipo *nVariable;* //Se declara variable que contendrá la referencia al objeto

//Algoritmo:

nVariable = ***new*** *MétodoConstructor*(); //Crear objeto

```
//Entorno:
    Yeti j;
//Algoritmo:
    j=new Yeti();
```



5. Crear un objeto o instancia

//Entorno:

tipo *nVariable;* //Se declara variable que contendrá la referencia al objeto

//Algoritmo:

nVariable = ***new*** *MétodoConstructor*(); //Crear objeto

¿Quién asigna la cantidad de memoria suficiente para ese objeto?

Operador **new**

¿Qué hace **new** exactamente?

- Asigna memoria para el objeto.
- Inicia los atributos de ese objeto a sus valores por defecto.
- Llama al constructor de la clase indicado al crear el objeto.

6. Constructores

CONSTRUCTOR

Es un método cuya función es iniciar nuevos objetos de su clase. Sólo puede ser llamado por el operador **new**.

- Cuando se crea un objeto, Java hace lo siguiente:
  - Asigna memoria para el objeto por medio del operador **new**
  - quien inicia los atributos de ese objeto a sus valores por defecto
  - y llama al constructor de la clase utilizado en la creación del objeto.
- El constructor, como el resto de métodos, puede estar sobrecargado
- Constructor por omisión o por defecto es el constructor sin parámetros
- Si en una clase no se implementa ningún constructor, siempre existirá el constructor por defecto.

6. Constructores

Los constructores deben cumplir unas especificaciones:

- ❖ Tiene el mismo nombre que la clase a la que pertenece.
- ❖ No se hereda.
- ❖ No puede retornar un valor.
- ❖ No puede ser declarado ***final***, ***static***, ***abstract***, ***synchronized*** o ***native***.

- ❑ El constructor será invocado cuando se cree un objeto por el operador new.
- ❑ El objeto será construido justo antes de iniciar la ejecución del constructor.
- ❑ En el cuerpo del constructor se podrá asignar valores a los atributos, invocar a métodos de la clase o bien llamar a métodos de otros objetos.

7. Destrucción de objetos

MÉTODO *finalize*

Se invoca automáticamente cada vez que se destruye un objeto.
Cuando un objeto se destruye:

- Se invoca al método **finalize**
- El recolector de basura (**garbage collector**) se encarga de eliminar el objeto y liberar los recursos que tuviera adjudicados.

Un objeto es eliminado automáticamente cuando se eliminan todas las referencias al mismo:

- o Bien porque se ha salido del ámbito en el que está declarado
- o Porque se quede sin referencia.
- o Bien porque se asigna **null** a la variable que lo referenciaba.

Si la aplicación Java termina sin que se eliminen los objetos será labor de S.O.

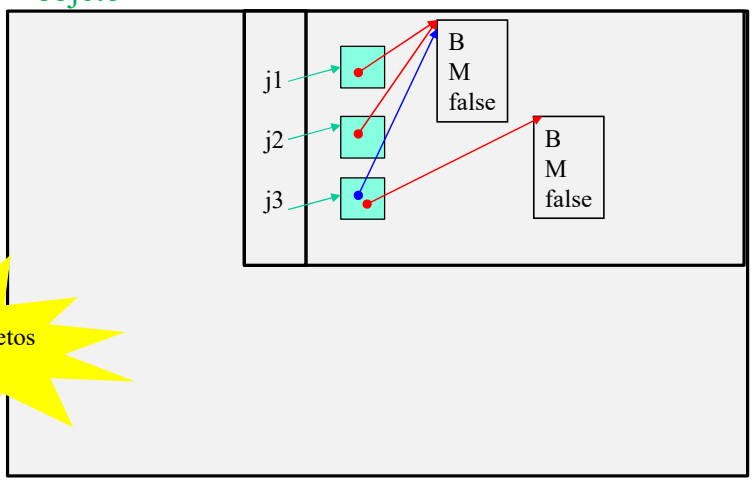
7. Destrucción de objetos

//Entorno:

Yeti j1,j2,j3; //Se declara variable que contendrá la referencia al objeto

//Algoritmo:

j1=new Yeti();
j2=j1;
j3=new Yeti();
j3=j2;



se crean 2 objetos
y se destruye 1

8. Asignación de objetos

¿Crear nueva referencia = Crear nuevo objeto?

¿Crear nueva referencia ➔ Crear nuevo objeto?

¿Crear nuevo objeto ➔ Crear nueva referencia?